Συστήματα Υπολογισμού Υψηλών Επιδόσεων
Project 3

Τσόγκας Παναγιώτης	**Νίκας Ιωάννης Ιάσων**	**Τσιαντός Δημήτριος**
ptsogkas@uth.gr	ioannikas@uth.gr	dtsiantos@uth.gr
3672	3771	3796

Νοέμβριος 2025

Περιεχόμενα

1 Πληροφορίες συστήματος **3**

2 Πειραματική μελέτη: Μέγιστο μέγεθος εικόνας και ακρίβεια **3**
2.1 Όρια μεγέθους εικόνας . . . 3
2.2 FMAD . . . 3
2.3 Ακρίβεια υπολογισμών . . . 4

3 Υλοποίηση πολλαπλών thread blocks **5**
3.1 Όρια μεγέθους εικόνας . . . 5

4 Πειραματική μελέτη με floats **6**
4.1 Μέγιστη διαφορά . . . 6
4.2 Χρόνοι CPU &GPU ανά μέγεθος εικόνας . . . 6

5 Πειραματική μελέτη με doubles **8**
5.1 Μέγιστη διαφορά . . . 8
5.2 Χρόνοι CPU &GPU ανά μέγεθος εικόνας . . . 8

6 Προσπελάσεις στην μνήμη και FLOPS **10**
6.1 Αναγνώσεις εικόνας και φίλτρου . . . 10
6.2 Λόγος προσπελάσεων προς FLOPs . . . 10

7 Εξάλειψη Divergence **10**

A Appendix **12**

1 Πληροφορίες συστήματος

Εκτελώντας το device query από τα cuda-samples μπορούμε να δούμε διάφορες πληροφορίες σχετικά με τις κάρτες γραφικών του συστήματος. Βλέπουμε πως είναι συνδεδεμένες τέσσερις Tesla K80 συσκευές (δύο ανά πλακέτα) με συνολικό μέγεθος global memory ανά συσκευή στα `11441 MBytes` αντί για `12 Gbytes` (`24 Gbytes` ανά πλακέτα). Αυτό συμβαίνει επειδή το ECC είναι ενεργοποιημένο και καταλαμβάνει ∼6.25% της συνολικής μνήμης, αφήνοντας ∼`11.25 GBytes` ανά συσκευή για τον χρήστη.

Memory Type	**Size**
Global Memory (VRAM)	$11\,441\,\mathrm{MB}$ (Approx. $11.2\,\mathrm{GB}$)
L2 Cache Size (Global)	$1536\,\mathrm{kB}$ (Approx. $1.5\,\mathrm{MB}$)
Shared Memory per Block	$48\,\mathrm{kB}$
Total Constant Memory (Device)	$64\,\mathrm{kB}$
Registers per Block	$65\,536$ (32-bit registers)

2 Πειραματική μελέτη: Μέγιστο μέγεθος εικόνας και ακρίβεια

2.1 Όρια μεγέθους εικόνας

Με την χρήση ενός block για την υλοποίηση του kernel, το οποίο περιλαμβάνει 1024 threads σε αντιστοίχιση ένα προς ένα με τα pixels, μπορούμε να προβλέψουμε (βάσει των δεδομένων του deviceQuery A) ότι η μέγιστη διάσταση της εικόνας που μπορούμε να επεξεργαστούμε είναι $W \times H = 1024$. Δεδομένου ότι $W = H$, για απλούστερη υλοποίηση, προκύπτει ότι $W = H = \sqrt{1024} = 32$. Το οποίο παρατηρείται και πειραματικά καθώς με πλάτος και ύψος παραπάνω από 32 παίρνουμε το παρακάτω μήνυμα σφάλματος :

```
CUDA Error:  invalid argument in Convolution2D.cu, line 235
```

2.2 FMAD

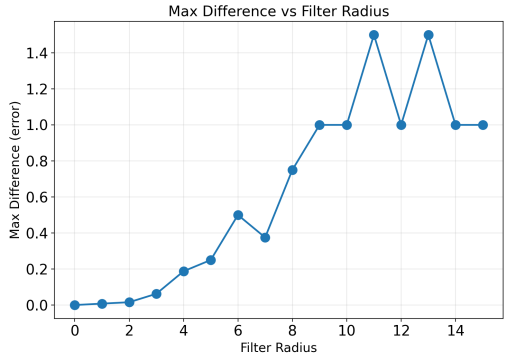
Στις δοκιμές μας παρατηρήθηκε σημαντική απόκλιση μεταξύ των αποτελεσμάτων του Host και του Device. Η απόκλιση αυτή εξαφανίστηκε με τη χρήση του -G κατά τη μεταγλώττιση. Από το nvcc –help επιβεβαιώσαμε ότι το συγκεκριμένο flag απενεργοποιεί τις βελτιστοποιήσεις, μεταξύ των οποίων βρίσκεται και η χρήση της fused multiply–add (FMAD) πράξης. Η πράξη αυτή επιτρέπει στο Device να εκτελεί πολλαπλασιασμό και πρόσθεση σε ένα μόνο instruction, με μεγαλύτερη ακρίβεια λόγω της μονής στρογγυλοποίησης. Παραδόξως, η διαφορά που παρατηρήσαμε οφείλεται τελικά στην αριθμητική ανακρίβεια του Host και όχι σε σφάλμα του Device όταν το FMAD είναι ενεργοποιημένο.

2.3 Ακρίβεια υπολογισμών

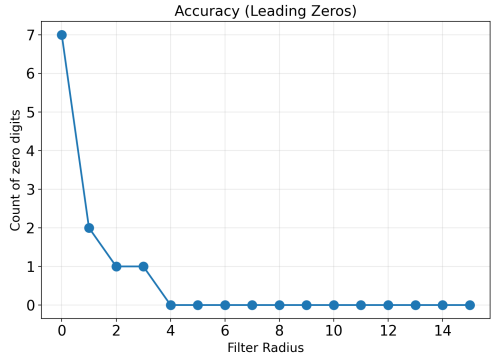
Για εικόνα με διαστάσεις 32×32 και δοκιμάζοντας φίλτρα με ακτίνα από 0 μέχρι 15 κάναμε τις εξής μετρήσεις για την ακρίβεια του προγράμματος:

Ακτίνα φίλτρου	**Μέγιστη διαφορά**
0	0
1	0.0078125
2	0.015625
3	0.0625
4	0.1875
5	0.25
6	0.5
7	0.375
8	0.75
9	1.0
10	1.0
11	1.5
12	1.0
13	1.5
14	1.0
15	1.0

Συμπεραίνουμε ότι καθώς αυξάνεται το μέγεθος του φίλτρου η ακρίβεια μειώνεται σημαντικά.



(α′) Μέγιστη διαφορά συναρτήσει της ακτίνας



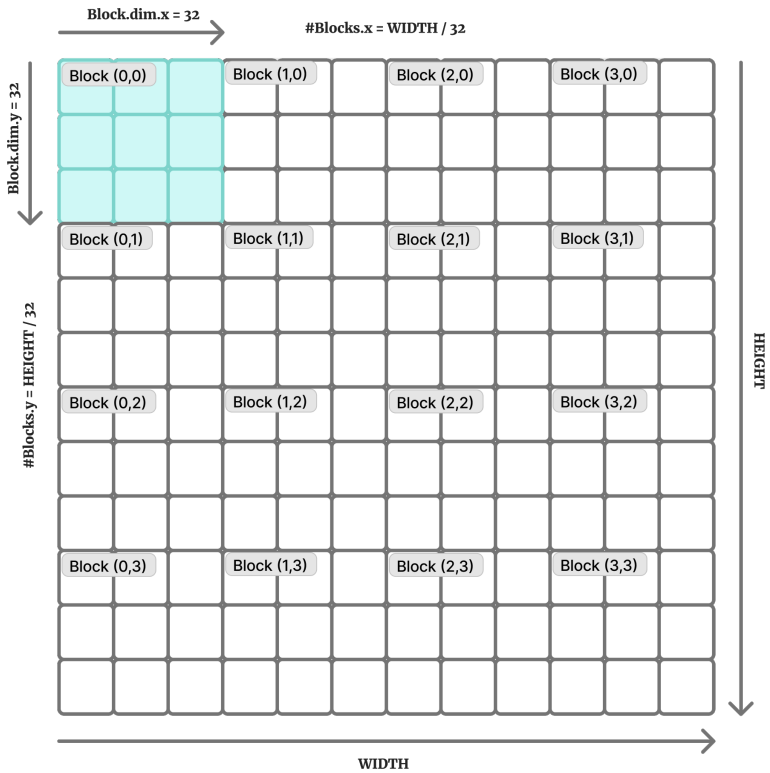
(β′) Μέγιστη διαφορά σε ψηφία

Σχήμα 1: Ανάλυση σφαλμάτων συναρτήσει της ακτίνας φίλτρου

3 Υλοποίηση πολλαπλών thread blocks

Για την υλοποίηση με πολλαπλά block χρησιμοποιούμε την δομή που παρουσιάζεται στο Σχήμα 2 για την οργάνωση των threads. Το κάθε thread εντοπίζει το pixel στο οποίο θα εργαστεί μέσω του παρακάτω indexing.

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
```



Σχήμα 2: Μορφή Grid

3.1 Όρια μεγέθους εικόνας

Θεωρητικά, η χρήση πολλαπλών blocks επιτρέπει να ζητήσουμε την εκτέλεση ενός kernel με grid έως $int_max \cdot 65536$, που αποτελεί το αυθαίρετο όριο της CUDA. Κάθε block έχει διαστάσεις $block_width = 32$, άρα $32 \times 32 = 1024$ threads, όπως αναφέραμε προηγουμένως. Συνεπώς, το θεωρητικό μέγιστο μέγεθος εικόνας που θα μπορούσε να δοθεί ως είσοδος είναι

$$(block_width \cdot int_max) \times (block_width \cdot int_max).$$

Παρόλα αυτά, στην πράξη το πρόγραμμα σταματάει να λειτουργεί πολύ νωρίτερα, καθώς το πραγματικό bottleneck δεν είναι τα όρια του grid, αλλά η διαθέσιμη μνήμη. Συγκεκριμένα, στο Device, όταν χρησιμοποιούμε τους πίνακες d_Input, d_Buffer και d_Output, η συνολική δεσμευμένη μνήμη ανέρχεται σε $3 \cdot W \cdot H \cdot$ sizeof(pixel), γεγονός που περιορίζει άμεσα το μέγιστο επιτρεπτό μέγεθος εικόνας. Έστω μνήμη 11997020160 bytes, απο deviceQuery A, έχουμε:

$$\text{TotalAlloc}_{\text{bytes}} = 3 \cdot \text{Width} \cdot \text{Height} \cdot \text{sizeof(PixelScalar)}_{\text{bytes}}$$

$$\text{Width} \cdot \text{Height} = \frac{11997020160_{\text{bytes}}}{4_{\text{bytes}} \cdot 3}$$

$$\text{Width} \cdot \text{Height} = 999751680, \quad (\text{ Width} = \text{Height} = \text{side})$$

$$\text{side} = \left\lfloor \sqrt{999751680} \right\rfloor = 31618$$

Αργότερα εφαρμόσαμε μια βελτιστοποίηση κατά την οποία χρησιμοποιούμε μόνο τους πίνακες d_Input και d_Buffer, ενώ ο αχρησιμοποίητος πίνακας d_Input επαναχρησιμοποιείται ως πίνακας output, εξοικονομώντας σημαντικό χώρο μνήμης. Συνεπώς το νέο όριο είναι :

$$\text{TotalAlloc}_{\text{bytes}} = 2 \cdot \text{Width} \cdot \text{Height} \cdot \text{sizeof(PixelScalar)}_{\text{bytes}}$$

$$\text{Width} \cdot \text{Height} = 1499627520, \quad (\text{ Width} = \text{Height} = \text{side})$$

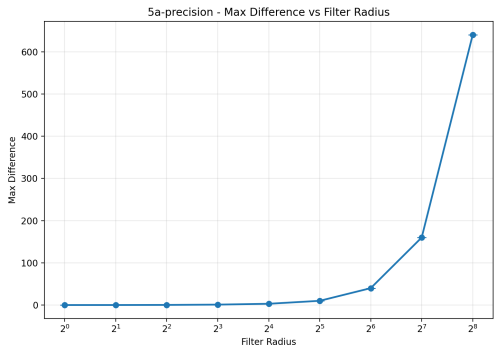
$$\text{side} = \left\lfloor \sqrt{1499627520} \right\rfloor = 38725$$

Στην πραγματικότητα πάνω στο Device μπορούμε να τρέχουμε τον αλγόριθμο ταυτόχρονα για #MPs · #ThreadsMPs, που είναι $13 \cdot 2048 = 26624_{pixels}$.

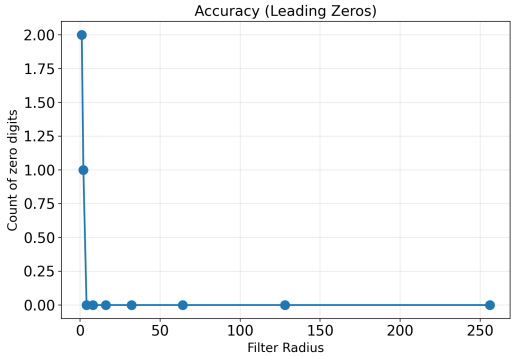
4 Πειραματική μελέτη με floats

4.1 Μέγιστη διαφορά

Στο πρόγραμμα παρατηρούμε μεγάλη απόκλιση της μέγιστης διαφοράς στο αποτέλεσμα μεταξύ του host και του device. Αυτό οφείλεται στον διαφορετικό τρόπο εκτέλεσης των εντολών στις δύο αρχιτεκτονικές. Ο επεξεργαστής τρέχει τις εντολές σειριακά ανά pixel, κάνοντας στρογγυλοποίηση ύστερα από κάθε floating point πράξη. Αντίθετα, η κάρτα γραφικών εκτελεί σε μία εντολή έναν πολλαπλασιασμό και μία πρόσθεση (FMAD 2.2), χωρίς να χάνει πολύ ακρίβεια.



(α′) Μέγιστη διαφορά συναρτήσει της ακτίνας.



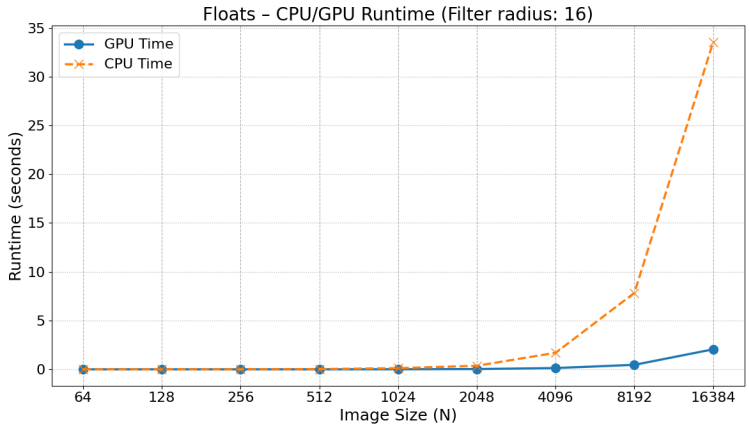
(β′) Μέγιστη διαφορά σε ψηφία.

Σχήμα 3: Ανάλυση σφαλμάτων συναρτήσει της ακτίνας φίλτρου.

Ακτίνα φίλτρου	1	2	4	8	16	32	64	128	256
Μέγιστη διαφορά	0.0078125	0.046875	0.25	1	3	10	40	160	640

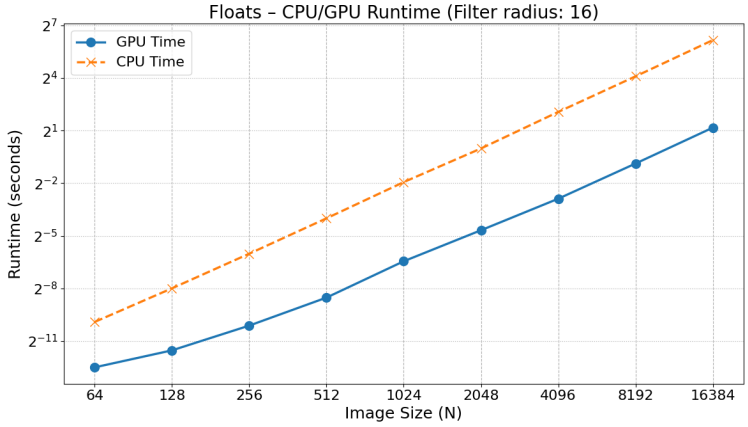
4.2 Χρόνοι CPU & GPU ανά μέγεθος εικόνας

Όσο αυξάνεται το μέγεθος της εικόνας, ο χρόνος εκτέλεσης τόσο στον επεξεργαστή όσο και στην κάρτα γραφικών αυξάνονται εκθετικά. Παρόλο που και τα δύο έχουν παρόμοιο ρυθμό αύξησης, η κάρτα γραφικών έχει πολύ μικρότερο χρόνο εκτέλεσης.



Σχήμα 4: Χρόνος εκτέλεσης με floats ανά μέγεθος εικόνας.

Μέγεθος εικόνας	**GPU avg (s)**	**GPU Std dev (s)**	**CPU avg (s)**	**CPU Std dev (s)**	**Speedup**
64×64	0.0001489	7.3401E-06	0.0004886	2.1329E-05	3.28×
128×128	0.0002829	6.5735E-06	0.0019377	6.0046E-05	6.85×
256×256	0.0007317	9.2622E-06	0.0074893	9.9947E-05	10.24×
512×512	0.0026406	2.3566E-05	0.0299085	0.0002121	11.33×
1024×1024	0.0094353	4.7866E-05	0.1218538	0.0029249	12.91×
2048×2048	0.0334919	0.0004511	0.3646621	0.0161425	10.89×
4096×4096	0.1328186	0.0011445	1.6972503	0.1270327	12.78×
8192×8192	0.4616295	0.0368957	7.8352702	0.2023358	16.97×
16384×16384	2.0517052	0.0949082	33.5328534	1.4619840	16.34×

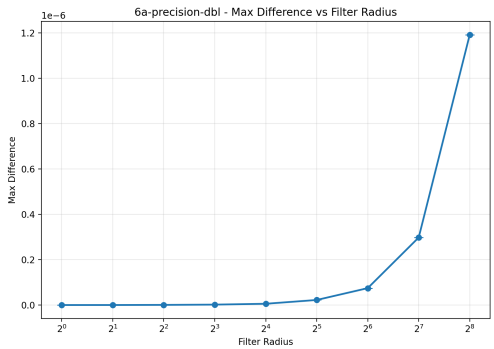


Σχήμα 5: Χρόνοι εκτέλεσης σε λογαριθμική κλίμακα.

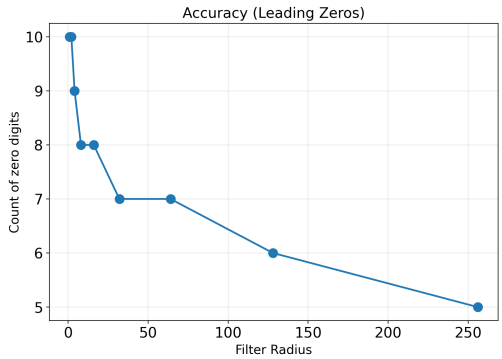
5 Πειραματική μελέτη με doubles

5.1 Μέγιστη διαφορά

Αλλάζοντας τον τύπο του pixel από float σε double βλέπουμε σημαντική μείωση στην μέγιστη διαφορά, καθώς ο double έχει καλύτερη ακρίβεια. Συγκεκριμένα, ενώ ο float χρησιμοποιεί 32 bits για την αναπαράσταση ενός αριθμού κινητής υποδιαστολής, ενώ ο double χρησιμοποιεί 64 bits. Αυτό σημαίνει ότι οι πράξεις που περιλαμβάνουν πολλαπλασιασμούς και προσθέσεις επηρεάζονται λιγότερο από σφάλματα στρογγυλοποίησης. Η χρήση double γίνεται σε περιπτώσεις όπου απαιτείται υψηλή ακρίβεια.



(α′) Μέγιστη διαφορά συναρτήσει της ακτίνας.

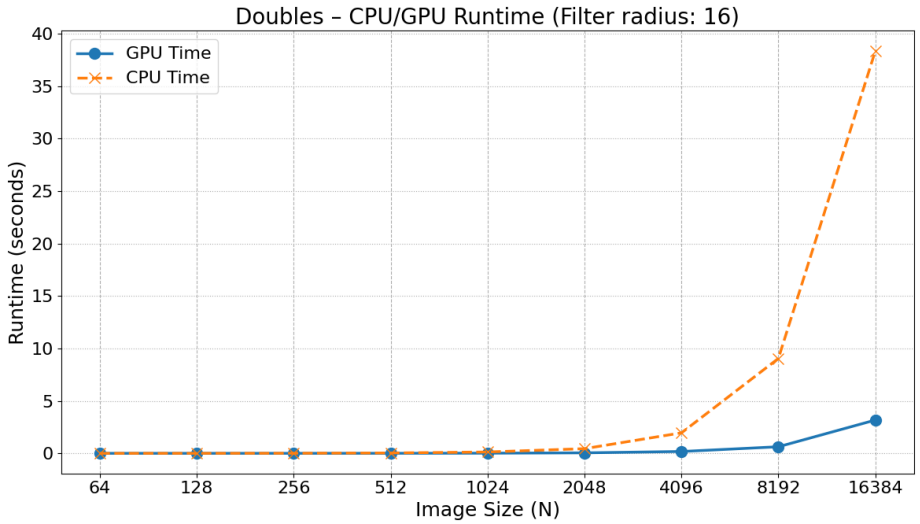


(β′) Μέγιστη διαφορά σε ψηφία.

Σχήμα 6: Ανάλυση σφαλμάτων συναρτήσει της ακτίνας φίλτρου.

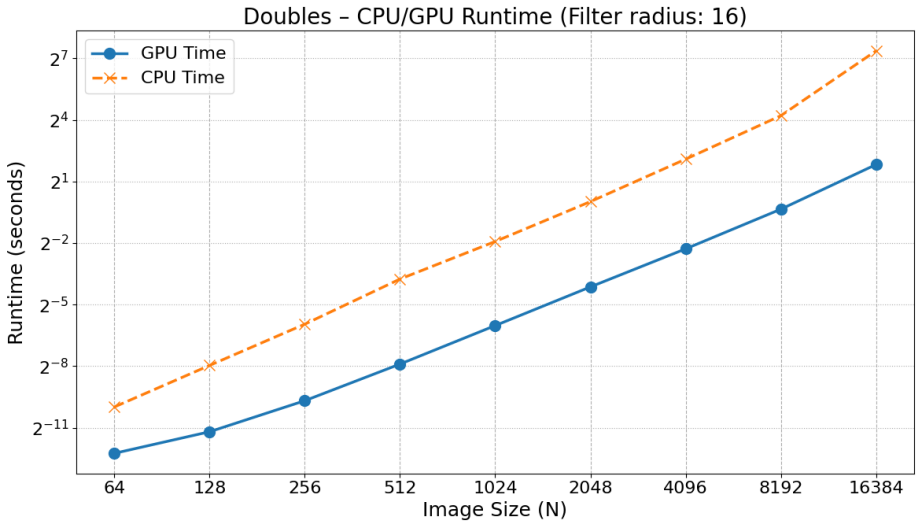
Ακτίνα φίλτρου	**Μέγιστη διαφορά**
1	1.4552E-11
2	8.7311E-11
4	5.82077E-10
8	1.862645E-09
16	5.587935E-09
32	2.2351742E-08
64	7.4505806E-08
128	2.98023224E-07
256	1.192092896E-06

5.2 Χρόνοι CPU & GPU ανά μέγεθος εικόνας



Σχήμα 7: Χρόνος εκτέλεσης προγράμματος σε CPU και σε GPU με χρήση doubles.

Μέγεθος εικόνας	**GPU avg (s)**	**GPU Std dev (s)**	**CPU avg (s)**	**CPU Std dev (s)**	**Speedup**
64×64	0.0001738	2.4738E-05	0.000525	1.4184E-04	3.02×
128×128	0.0003289	5.8585E-06	0.0019992	4.6557E-05	6.08×
256×256	0.0011041	2.4118E-05	0.0082182	1.44198E-04	7.44×
512×512	0.0037084	2.2136E-04	0.034054	0.0033225	9.18×
1024×1024	0.0131907	8.6974E-04	0.1278087	0.0151046	9.69×
2048×2048	0.0464747	2.2383E-03	0.4439873	0.0413447	9.55×
4096×4096	0.1740196	1.4832E-03	1.9524106	0.101432	11.22×
8192×8192	0.6261251	3.5972E-02	9.0471454	0.426256	14.45×
16384×16384	3.172902	0.442676	38.3644301	1.369290	12.09×



Σχήμα 8: Χρόνοι εκτέλεσης σε λογαριθμική κλίμακα.

6 Προσπελάσεις στην μνήμη και FLOPS

6.1 Αναγνώσεις εικόνας και φίλτρου

Τα στοιχεία της εικόνας που βρίσκονται σε απόσταση μεγαλύτερη από την ακτίνα του φίλτρου από τα άκρα διαβάζονται τόσες φορές όσα είναι και τα pixels του filter. Αυτό συμβαίνει επειδή, κατά τη διαδικασία της συνέλιξης, κάθε φορά που το filter εφαρμόζεται σε ένα pixel, όλα τα στοιχεία που βρίσκονται εντός απόστασης filter_radius από το κέντρο του filter προσπελαύνονται. Έτσι, κάθε εσωτερικό pixel συμμετέχει στο συνολικό πλήθος προσπελάσεων όσες φορές αντιστοιχούν στο εμβαδόν του kernel.

Τα στοιχεία του filter, συγκεκριμένα το κέντρο του, προσπελαύνονται τόσες φορές όσες και τα pixels της εικόνας, καθώς συμμετέχουν σε κάθε εφαρμογή του φίλτρου. Τα υπόλοιπα στοιχεία πέρα από το κεντρικό του filter, όμως, δεν προσπελαύνονται όταν το φίλτρο βρίσκεται σε απόσταση μικρότερη από filter_radius από τα άκρα της εικόνας. Αξίζει να σημειωθεί ότι, οι περιπτώσεις λιγότερων προσπελάσεων δεν ισχύουν στην υλοποίηση με padding.

6.2 Λόγος προσπελάσεων προς FLOPs

Κοιτάζοντας τον κώδικα παρατηρούμε πως κατά την εκτέλεση της συνέλιξης έχουμε δύο προσπελάσεις στην μνήμη, μία για την τιμή του pixel και μία για την τιμή του φίλτρου. Ύστερα πολλαπλασιάζουμε αυτούς τους δύο αριθμούς κινητής υποδιαστολής και προσθέτουμε το αποτέλεσμα στο σύνολο.

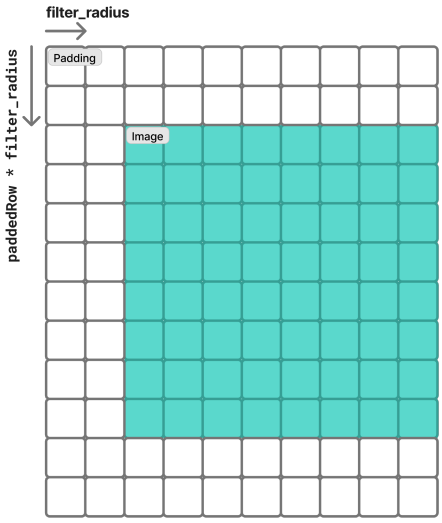
```
for (k = -filterR; k <= filterR; k++) {
    int d = x + k;

    if (d >= 0 && d < imageW)
        // two accesses, two FLOPs
        sum += h_Src[y * imageW + d] * h_Filter[filterR - k];
}
```

Εφόσον έχουμε δύο προσπελάσεις στην μνήμη και δύο πράξεις κινητής υποδιαστολής, ο λόγος memory accesses προς FLOPs είναι 1.

7 Εξάλειψη Divergence

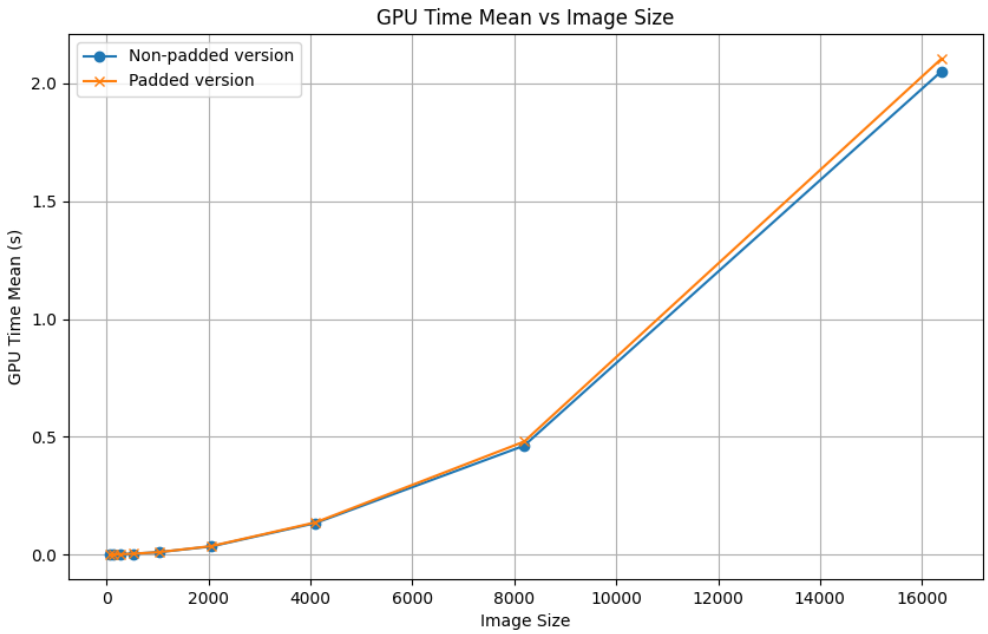
Για την επίλυση του divergence χρησιμοποιήσαμε padding όπως φαίνεται στο σχήμα 9. Παρακάτω βλέπουμε τις εκτελέσεις για padded και non-padded εκδόσεις.

Στην την υλοποίηση του padded κώδικα, για να σώσουμε μνήμη, χρησιμοποιήσαμε αρχικά μια συνάρτηση cudaTransferPadded η οποία είχε αρκετό overhead λόγω των πολλαπλών κλήσεων της cudaMemcpy αργότερα αντικαταστήσαμε αυτή την συνάρτηση με την cudaMemcpy2D, που μπορεί να χρησιμοποιηθεί για αυτή την λειτουργία.



Σχήμα 9: Padded image για filter_radius = 2

```
void cudaTransferPadded (int xPad, int yPad, PixelScalar *device,
                         PixelScalar *host, int arrayWidth,
                         int arrayHeight, cudaMemcpyKind kind)
{
    int paddedRow = (xPad << 1) + arrayWidth;
    int startOffset = (paddedRow * yPad) + xPad;
    PixelScalar *curRowStartD = device + startOffset;
    PixelScalar *curRowStartH = host;
    for (int i = 0; i < arrayHeight; i++) {
        if (kind == cudaMemcpyHostToDevice)
            cudaMemcpy(curRowStartD, curRowStartH, arrayWidth * sizeof(PixelScalar), kind);
        else
            cudaMemcpy(curRowStartH, curRowStartD, arrayWidth * sizeof(PixelScalar), kind);
        curRowStartD += (xPad << 1) + arrayWidth;  //* Skips side padding
        curRowStartH += arrayWidth; //* Go next line on host
    }
}
```



Σχήμα 10: Σύγκριση χρόνου εκτέλεσης χωρίς και με padding.

Παρατηρούμε πως οι χρόνοι εκτέλεσης δεν διαφέρουν σε μεγάλο βαθμό. Αυτό πιθανόν να οφείλεται στο γεγονός ότι παρόλο που κερδίζουμε από την εξάλειψη του φαινομένου του divergence χάνουμε από τις πιο περίπλοκες μεταφορές που απαιτεί ο padded πίνακας. Μετά από επιπλέον δοκιμές σε δικό μας σύστημα παρατηρήθηκε η επιθυμητή διαφορά ανάμεσα στους χρόνους.

Filter Radius	GPU Time (Non-padded)	GPU Time (Padded)
128	0.989189	0.920196
256	1.354624	1.228230
512	2.150286	1.900903
1024	3.711623	3.193815
2048	6.827488	5.756322
4096	13.053341	10.914864

Table 1: Ετκέλεση σε σύστημα με `RTX 3060 12GB` σε πίνακα 16384.

Filter Radius	GPU Time (Non-padded)	GPU Time (Padded)
128	3.823996	3.604211
2048	27.961308	23.122805

Table 2: Ετκέλεση σε σύστημα με `RTX 3060 12GB` σε πίνακα 32768.

A Appendix

Αποτέλεσμα του deviceQuery

```
./deviceQuery Starting...

 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 4 CUDA Capable device(s)

Device 0: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (13) Multiprocessors, (192) CUDA Cores/MP:     2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536),
                                                 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 6 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device
     simultaneously) >

Device 1: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (13) Multiprocessors, (192) CUDA Cores/MP:     2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536),
                                                 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
```

```
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 7 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device
     simultaneously) >

Device 2: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (13) Multiprocessors, (192) CUDA Cores/MP:     2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536),
                                                 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 132 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device
     simultaneously) >

Device 3: "Tesla K80"
  CUDA Driver Version / Runtime Version          11.4 / 11.5
  CUDA Capability Major/Minor version number:    3.7
```

```
  Total amount of global memory:                 11441 MBytes (11997020160 bytes)
  (13) Multiprocessors, (192) CUDA Cores/MP:     2496 CUDA Cores
  GPU Max Clock rate:                            824 MHz (0.82 GHz)
  Memory Clock rate:                             2505 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 1572864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(65536), 2D=(65536, 65536),
                                                 3D=(4096, 4096, 4096)
  Maximum Layered 1D Texture Size, (num) layers  1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(16384, 16384), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  2048
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Compute Preemption:            No
  Supports Cooperative Kernel Launch:            No
  Supports MultiDevice Co-op Kernel Launch:      No
  Device PCI Domain ID / Bus ID / location ID:   0 / 133 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device
     simultaneously) >
> Peer access from Tesla K80 (GPU0) -> Tesla K80 (GPU1) : Yes
> Peer access from Tesla K80 (GPU0) -> Tesla K80 (GPU2) : No
> Peer access from Tesla K80 (GPU0) -> Tesla K80 (GPU3) : No
> Peer access from Tesla K80 (GPU1) -> Tesla K80 (GPU0) : Yes
> Peer access from Tesla K80 (GPU1) -> Tesla K80 (GPU2) : No
> Peer access from Tesla K80 (GPU1) -> Tesla K80 (GPU3) : No
> Peer access from Tesla K80 (GPU2) -> Tesla K80 (GPU0) : No
> Peer access from Tesla K80 (GPU2) -> Tesla K80 (GPU1) : No
> Peer access from Tesla K80 (GPU2) -> Tesla K80 (GPU3) : Yes
> Peer access from Tesla K80 (GPU3) -> Tesla K80 (GPU0) : No
> Peer access from Tesla K80 (GPU3) -> Tesla K80 (GPU1) : No
> Peer access from Tesla K80 (GPU3) -> Tesla K80 (GPU2) : Yes

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 11.4,
CUDA Runtime Version = 11.5, NumDevs = 4, Device0 = Tesla K80,
Device1 = Tesla K80, Device2 = Tesla K80, Device3 = Tesla K80
Result = PASS
```